

Instituto Tecnológico Superior de Rio verde SLP.

TEMA:

Semana 13

ALUMNO:

Brayan Ivanov Martínez Martínez

MATERIA:

programacion

DOCENTE:

Jesus collazo

GRUPO

2ª A

**Fecha
14/05/2026**

Introducción

La programación orientada a objetos permite hacer programas más organizados y fáciles de entender. En esta práctica se creó un sistema de empleados usando temas importantes de POO en C++, como herencia, polimorfismo, clases abstractas e interfaces. Gracias a esto, el programa puede manejar diferentes tipos de empleados de una forma más sencilla y ordenada.

La clase abstracta funciona como una base para otras clases. No se pueden crear objetos directamente de ella, pero ayuda a compartir información y funciones comunes. En este proyecto, la clase “Empleado” sirvió para guardar datos y métodos que todos los empleados necesitan.

También se utilizó una interfaz llamada “IResponsable”. Una interfaz obliga a las clases a realizar ciertas acciones, en este caso generar reportes. La diferencia entre una clase abstracta y una interfaz es que la clase abstracta puede tener atributos y métodos ya hechos, mientras que la interfaz solo indica qué métodos deben existir.

Todos estos conceptos ayudan a que el programa sea más flexible y más fácil de modificar en el futuro. Además, permiten reutilizar código y agregar nuevas funciones sin tener que cambiar todo el sistema.

Desarrollo

El sistema realizado representa una empresa tecnológica donde trabajan diferentes tipos de empleados y cada uno tiene tareas distintas. El programa permite agregar empleados, mostrar su información, realizar actividades y crear reportes usando un menú dinámico.

La clase principal del sistema es “Empleado”, la cual se utilizó como clase abstracta. De esta clase se derivan las siguientes clases:

- Programador
- AdministradorRed
- SoporteTecnico

Cada una de estas clases redefine el método trabajar(), haciendo que cada empleado realice acciones diferentes dependiendo de su puesto dentro de la empresa.

También se utilizó la interfaz “IResponsable”, que obliga a todos los empleados a implementar el método generarReporte(). Gracias a esto, cada empleado puede generar un reporte diferente según las actividades que realiza.

El programa también usa polimorfismo mediante un vector de punteros tipo Empleado. Esto permite guardar distintos tipos de empleados dentro de la misma estructura y trabajar con ellos de forma más sencilla y organizada.

Diseño de clases

Clase abstracta:

Empleado

Atributos:

- nombre
- edad
- salario
- bonoMensual

Métodos:

- mostrarInformacion()
- trabajar()
- realizarActividad()
- calcularSalarioTotal()

Interfaz:

IResponsable

Método:

- generarReporte()

Clases derivadas:

- Programador
- AdministradorRed
- SoporteTecnico

Relación entre clases e interfaces

Las clases Programador, AdministradorRed y SoporteTecnico heredan de la clase abstracta Empleado para aprovechar los atributos y métodos que todos los

empleados tienen en común. De esta forma se evita repetir código y el programa queda más ordenado.

Además, estas clases implementan la interfaz IResponsable, la cual obliga a que cada empleado tenga el método generarReporte(). Gracias a esto, cada tipo de empleado puede generar un reporte diferente según el trabajo que realiza.

Con esto, todos los empleados comparten una misma base, pero cada uno tiene funciones y comportamientos distintos dependiendo de su puesto dentro de la empresa.

Ejemplo de ejecución

```
Nombre: luka
Edad: 23
Salario Base: $1500
Bono Mensual: $300
Salario Total: $1800
Nombre: paco
Edad: 55
Salario Base: $2300
Bono Mensual: $400
Salario Total: $2700
Nombre: amanda
Edad: 20
Salario Base: $1900
Bono Mensual: $350
Salario Total: $2250
=== MENU ===
1. Registrar Programador
2. Registrar Administrador de Red
3. Registrar Soporte Tecnico
4. Mostrar empleados
5. Ejecutar actividades
6. Generar reportes
7. Salir
Seleccione una opcion: 5

=== ACTIVIDADES ===
luka esta desarrollando codigo del sistema.
paco esta monitoreando servidores y red.
amanda esta atendiendo incidencias tecnicas.

=== MENU ===
1. Registrar Programador
2. Registrar Administrador de Red
3. Registrar Soporte Tecnico
4. Mostrar empleados
5. Ejecutar actividades
6. Generar reportes
7. Salir
Seleccione una opcion: 6

=== REPORTES ===
Reporte: avances del desarrollo del software.
Reporte: estado actual de la red y servidores.
Reporte: tickets e incidencias atendidas.
```

Después de ingresar los datos de cada empleado, como nombre, edad, salario y bono mensual, el programa mostró la información registrada de todos los trabajadores. Después se pudieron visualizar las actividades que realiza cada empleado y también los reportes relacionados con su trabajo según el puesto que tienen dentro de la empresa.

Explicación técnica

¿Cómo se implementó la clase abstracta?

La clase abstracta se implementó usando la clase Empleado. Esta clase contiene un método virtual puro llamado trabajar(), lo que evita que se puedan crear objetos directamente de ella. Además, obliga a las clases hijas a crear su propia versión de ese método.

¿Cómo se implementó la interfaz?

La interfaz se implementó mediante la clase IResponsable, la cual solo contiene métodos virtuales puros. Las clases derivadas tuvieron que implementar el método generarReporte(), haciendo que cada empleado genere un reporte diferente dependiendo de su función.

¿Qué ventajas tiene usar interfaces?

Las interfaces ayudan a que varias clases tengan comportamientos obligatorios en común. También permiten que el programa sea más organizado, flexible y fácil de mantener o mejorar en el futuro.

¿Cómo ayuda el polimorfismo en este sistema?

El polimorfismo ayudó a manejar diferentes tipos de empleados usando un mismo vector de punteros tipo Empleado. Gracias a esto, el programa puede trabajar con todos los empleados de forma general, pero cada uno ejecuta acciones diferentes dependiendo de su puesto al usar el método trabajar().

Conclusiones

Durante esta práctica hubo algunas dificultades al momento de usar clases abstractas, interfaces y polimorfismo, sobre todo al trabajar con punteros y métodos virtuales. Al principio fue un poco complicado entender cómo funcionaban juntos estos conceptos dentro del programa.

Aun así, la práctica ayudó a comprender mejor cómo funciona la programación orientada a objetos y cómo se pueden crear sistemas más ordenados y fáciles de ampliar. También permitió entender la importancia de reutilizar código y separar las responsabilidades de cada clase para que el programa sea más claro y sencillo de mantener.

Además, este tipo de estructura puede utilizarse en sistemas reales, por ejemplo para administrar empleados, usuarios o departamentos dentro de una empresa, ya que permite manejar diferentes comportamientos usando una misma base general.

